Case Study: Technical Limitations of Low-Code Development

System Evaluation & Constraint Analysis in Enterprise Application Building

TARGET SYSTEM	PLATFORM	DOMAIN	AUTHOR
Hotel Booking System	Zoho Creator Platform	Enterprise Systems / LCNC	[Vinnarasan J / AIML]

1. AIM OF THE CASE STUDY

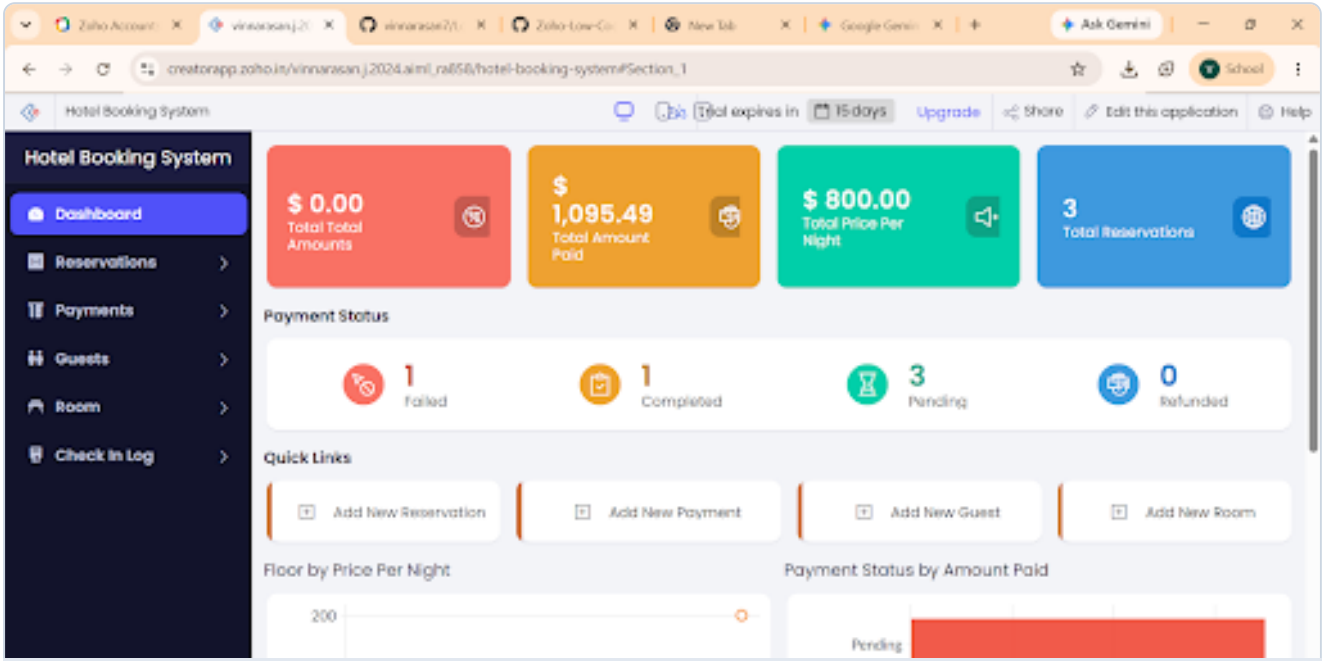
Objective: To systematically evaluate, analyze, and document the capabilities and architectural limitations of Low-Code Application Platforms (LCAP) using a real-world enterprise implementation (Hotel Booking System on Zoho Creator). The study focuses on evaluating data model restrictions, visual customization limits, workflow automation boundaries, and enterprise governance constraints.

2. METHODOLOGY & PROCEDURE

The evaluation was executed in a structured four-phase workflow to measure real-world performance against enterprise system requirements:

1. **Schema & Entity Design:** Created core business tables including Reservations, Payments, Guests, Rooms, and Check-In Logs using native low-code drag-and-drop builders.
2. **Dashboard & Interface Configuration:** Built metric widgets, quick-link navigation cards, dynamic filter bars, and status breakdown reports.
3. **Workflow & Business Logic Integration:** Formulated automated validation status checks (Verified, Pending, Failed) and payment calculation rules.
4. **Limitation & Constraint Auditing:** Evaluated system performance when attempting advanced dynamic UI layouts, custom complex data transformations, and external API integrations.

3. PRIMARY DASHBOARD IMPLEMENTATION



4. OPERATIONAL MODULE EVALUATION

During the implementation of specialized sub-modules, specific visual, workflow, and data structure constraints were identified across verification logging and guest directory views.

4.1 Check-In Verification Log View

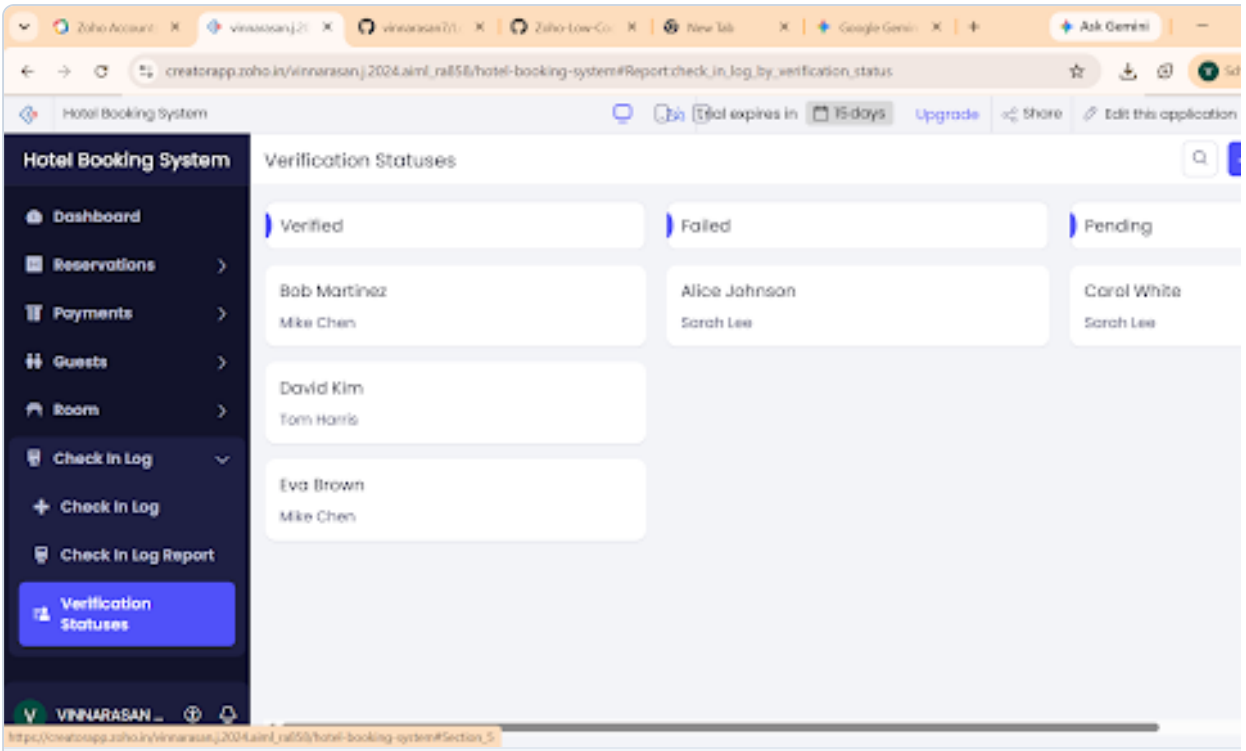


Figure 2.1: Kanban-style Verification Status interface categorizing guest records by verification status (Verified, Failed, Penc

4.2 Guest Information Directory

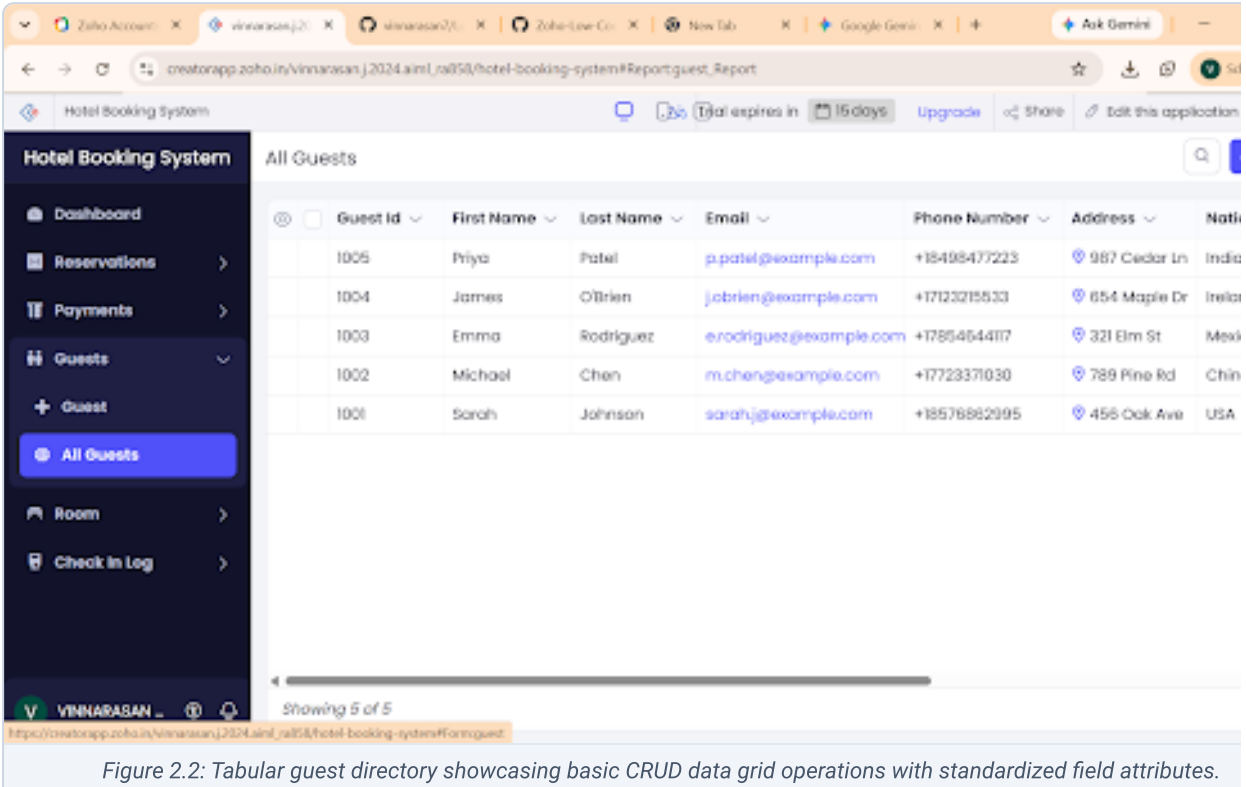


Figure 2.2: Tabular guest directory showcasing basic CRUD data grid operations with standardized field attributes.

5. CASE STUDY RESULTS & ANALYSIS

The practical development of the Hotel Booking System provided tangible data regarding where low-code platforms excel and where custom traditional code becomes necessary.

Evaluation Dimension	Platform Capability	Identified Limitation	Severity
UI / UX Design	Rapid drag-and-drop widget placement	Inflexible styling; restricted custom CSS/JS insertion	Medium
Complex Business Logic	Basic Deluge script triggers	High latency with multi-nested mathematical models	High
Data Architecture	Built-in relational lookup tables	Rigid ORM mapping; restricted raw SQL optimization	High
Integrations	Pre-built REST connectors	Rate limits on external webhooks and custom OAuth flows	Medium
Vendor Lock-in	Cloud-hosted rapid deployment	Non-portable proprietary code (Deluge/Low-code structure)	High

6. KEY TECHNICAL LIMITATIONS SUMMARY

- **Architectural Rigidity:** System capabilities are strictly bounded by pre-built component libraries. Extending functionality beyond platform primitives requires cumbersome workarounds.
- **Performance Bottlenecks:** Aggregated calculations on dynamic data sets exhibit noticeable performance overhead compared to direct SQL queries or optimized custom backends.
- **Governance & Version Control:** Native platforms lack granular Git-based version control, making parallel multi-developer branching and rollbacks difficult.
- **Cost Scalability:** Licensing models tie cost directly to user seats or record counts, leading to exponential cost spikes during business scaling.

7. CONCLUSION

> Low-code platforms like Zoho Creator significantly accelerate initial MVP deployment and standard CRUD applications (as demonstrated by the Hotel Booking System). However, for mission-critical enterprise systems requiring complex backend algorithms, highly custom user experiences, and multi-cloud portability, traditional code architectures remain essential.